# Crane Predictive Maintenance — Wiring, PCB Front-end, CubeMX Project & Starter Firmware Repo

**Owner:** Pragvansh Dharap **Contents (this document)** 1. Detailed wiring & PCB front-end schematic for ADS131E08 + IEPE & Bridge front-ends (textual + reference circuits + BOM) 2. PCB layout & EMC / grounding guidelines 3. Preconfigured STM32CubeMX project scaffold (SPI + DMA) — configuration steps and generated `main.c` skeleton 4. Starter firmware repository layout and **complete** example source files (drivers, FreeRTOS tasks, DMA handling, CMSIS-DSP FFT, feature extraction, demo) 5. A sample CSV file (device channel map & config) — content included 6. Next steps & testing checklist

---

## 1. Wiring & PCB Front-End Schematic (reference)

### Goals & constraints

- ADS131E08 is an 8-channel simultaneous ΔΣ ADC (24-bit) that talks over SPI. It expects differential inputs (±Vref range) and common-mode within device limits. We design two front-ends:
- **IEPE front-end** for accelerometers (AC-coupled, constant-current excitation).
- **Bridge front-end** for strain gauges / load cells (precision excitation, INA, anti-aliasing).
- System power: **24 VDC input** (crane), regulated to 5 V and 3.3 V. Analog domain uses clean 3.3 V ref for ADC.
- Isolation: digital isolator on SPI lines or isolated ADCs if needed near motors.

### High-level block connections

```
24V_IN -> Power Conditioning -> 5V, 3.3V, isolated 3.3V (if used)
Sensors: IEPE / Bridge / RTD / Current -> Front-end PCB -> ADS131E08 -> SPI ->
STM32 (SPI+DMA)
```

### Connector pinouts (suggested)

- **J1 (Power IN)**: 24V+, GND, chassis Earth
- **J2 (Sensors - IEPE / Vibration bank)**: For each accel: +IEPE, - (shield/GND), shield
- **J3 (Sensors - Bridge / Strain bank)**: Each bridge: EX+, EX-, SIG+, SIG-, SHIELD
- **J4 (Digital comms)**: CAN_H, CAN_L, RS485_A, RS485_B, ETH_Tx/Rx (if used)
- **J5 (Service)**: USB-UART (TTL), SWD pins, Boot0

---

## 1.1 IEPE Front-End (per channel)

**Purpose:** power IEPE accelerometer with constant current and provide AC-coupled conditioned output to ADC differential input.

**Reference circuit (single channel)** - Supply: +24V (or +18..+28V) -> local LDO to +12V or +5V depending on sensor spec (many IEPE use 18-30V, but some sensors accept 18–28V). For safety, we provide a regulated +24V to IEPE connectors. - **Constant current source:** LM334 (adjustable current source) configured for 4 mA. Alternative: discrete transistor current source with op-amp. - **Connection:** - +24V -> LM334 -> sensor +IESupply - Sensor output is AC-coupled on the signal lead via 1uF (C1) -> series resistor R_in (47Ω) -> input differential divider to make bipolar around mid-rail. - **AC coupling & bias removal:** Use capacitor C1 in series then bias with resistor network to create a mid-supply reference (Vbias = Vref/2 using two 100k resistors). - **Gain / anti-aliasing:** A differential input op-amp stage (OPA333/OPA378 or OPA2140) configured as differential amplifier with gain of 1–10 and a 2nd-order low-pass (Sallen-Key or diff-active) with cutoff fc = 0.45 * fs. - **Differential output** -> ADS131E08 input pins INxP / INxN.

**Parts & example values:** - LM334 (IEPE current source) + Rset choose for 4 mA (R = 1.2V / I ≈ 300Ω) - C1 = 1 µF, 50 V film (AC coupling) - Rbias1 = Rbias2 = 100k → Vbias = Vref/2 - Input series resistor R_in = 47Ω - Differential amp: INA149 or discrete op-amp pair - Anti-aliasing: R=10k, C=~1.6 nF for fc ≈ 10 kHz if fs=25 kSPS.

**Notes & reasoning:** - Use LM334 because it's a simple, rugged current source for IEPE sensors in field devices and tolerates 24V. - When IEPE sensors are connected/disconnected hot, the current source should be protected (series resistor + inrush limit) and the interface must clamp transients with TVS. - Many commercial IEPE accelerometers instead prefer an IEPE power module; for prototyping, LM334 commonly suffices.

---

## 1.2 Bridge Front-End (strain gauge / load cell)

**Purpose:** excite the bridge, amplify the differential mV-level output, drive ADC inputs with low noise, and provide RTD/thermistor support.

**Reference circuit (single bridge channel)** 1. **Bridge excitation** - Precision voltage reference: **REF5045** or ADR4525 for 2.5 V excitation. Use low-noise LDO to supply reference and excitation. - Bridge EX+ = Vref, Bridge EX- = GND (or excite with 2.5V differential). 2. **Instrumentation amplifier** - INA333 (rail-to-rail, low-noise) configured for gain G = 100–1000 depending on bridge sensitivity and ADC FS. - Gain resistor Rg computed by INA datasheet. 3. **Output filter** - 2nd-order anti-alias low-pass (fc = 0.45*fs). Passive RC following INA, or active 2-pole. 4. Differential level shift / offset** - Because ADS131E08 expects differential inputs, drive INxP = Vbias + vout/2 ; INxN = Vbias - vout/2 . Use buffered midrail Vbias (Vref/2) as common-mode.

**Example values:** - Bridge sensitivity 2 mV/V, Vex = 2.5V → full-scale bridge = 5 mV. With G=500 → 2.5 Vpp into ADC. - INA333 with Rg ≈ 49.9Ω for G ≈ 500 (consult datasheet). Use low-noise layout. - C_out filter: for fs=2 kSPS, fc ≈ 900 Hz; choose R=1k, C=~180 nF.

**RTD support** - Use MAX31865 or similar RTD ADC front-end if you prefer SPI-to-RTD. Alternatively, excite RTD with constant current (e.g., 1 mA via REF200 or constant source) and measure across ADC.

**Protection & diagnostics** - Use series resistors, TVS diodes, and input clamps to protect INA and ADC inputs from open-bridge or shorted conditions.

---

### 1.3 ADS131E08 connections & SPI wiring

- **Power:** VDD = 3.3 V (clean LDO), AVDD = 3.3 V analog domain.
- **REF:** Use clean reference (ADS internal OR external REF); ensure ADC Vref stable (0.01% drift).
- **AINs:** connect per-channel differential outputs from front-end to IN0P/IN0N ... IN7P/IN7N.
- **DRDY:** ADS131E08 has DRDY pin indicating new conversion. Wire DRDY to an EXTI pin on the STM32 for precise timestamping and to trigger DMA transfers.
- **CLK:** ADS can be clocked from MCU or local oscillator. If multiple ADCs need sync, use a master clock and/or use DRDY sync.
- **SPI:** SCLK, MOSI (SDI), MISO (SDO), CS. Put SPI MOSI if required for config writes.
- **RESET / PWDN:** optional reset line to MCU GPIO.

**SPI & DMA strategy:** - Use SPI in slave-master mode (MCU master), configure DMA to read SPI RX into double buffers. Use DRDY interrupt to trigger DMA transfer of one sample frame (ADS131E08 supports PWD/DRDY toggling to indicate data ready; read entire N-channel frame per DRDY).

---

## 2. PCB Layout & EMC / Grounding Guidelines

- **Partition analog & digital** zones. Place ADS131E08 and front-end op-amps in analog zone; microcontroller and regulators in digital zone.
- **Star ground**: single point at power entry. Connect sensor shields to chassis ground, not analog ground.
- **Guard rings** around high impedance nodes (bridge amp inputs).
- **Short traces** on differential pairs; keep matched length for INxP/INxN.
- **Decoupling**: 100 nF + 10 µF near all LDOs and ADC.
- **Power filtering**: Pi-filters for sensor input lines, common-mode chokes for long sensor cables.
- **Thermal relief** around connectors to avoid mechanical stress.
- **Silkscreen sensor IDs** and test points for each ADC channel.

---

## 3. Preconfigured STM32CubeMX Project (SPI + DMA) — Steps & main.c skeleton

### Target MCU

- **STM32H743ZI** (H7 family) — recommended for DSP + ML.

### CubeMX config (steps)

1. Create new project -> select STM32H743ZI.
2. Configure RCC: HSE (if present) -> PLL -> SYSCLK 400 MHz (or recommended stable frequency). Configure AHB/APB prescalers.
3. Enable Peripherals:
4. **SPI1** -> Mode: Full-Duplex Master, set Baud Rate Prescaler to support ADS SPI (max ADS SCLK ~5 MHz? check datasheet; set safely e.g., 5–10 MHz).
5. **GPIO**: DRDY pin as EXTI (FALLING EDGE), CS pin as GPIO output, RESET as GPIO output.
6. **DMA**: for SPI1_RX, Circular mode, two buffers (half/full transfer interrupts).
7. **TIM2**: free-running microsecond timer for timestamp capture.
8. **FATFS** -> SDIO for microSD.
9. **USART1** for service CLI.
10. **FreeRTOS** middleware.
11. Generate code (CubeIDE / Makefile project).

### main.c skeleton (auto-generated with additions)

(Only brief skeleton; full file in canvas repo)

```
/* main.c additions */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
  if (GPIO_Pin == DRDY_PIN) {
    // Arm DMA read sequence for SPI to fetch ADS frame
    // Option: use SPI Rx DMA in circular to continuously stream data
  }
}

void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef *hspi) {
  // Called on DMA full transfer
  // Push buffer pointer to queue for DSP processing
}

void start_acquisition() {
  // Assert CS low, start SPI DMA transfer sized to ADS frame length
}
```

Notes: the exact SPI frame length = bytes per sample * channels + status bytes — consult ADS131E08 datasheet. Typically read N*3 bytes for 24-bit samples + status.

---

## 4. Starter Firmware Repo (file tree + key files)

```
crane-pm-firmware/
├─ .gitignore
```

```
├─ README.md
├─ CubeIDE_Project/    # CubeMX generated project files
├─ firmware/
│  ├─ Inc/
│  │  ├─ ads131e08.h
│  │  ├─ iepe_frontend.h
│  │  ├─ bridge_frontend.h
│  │  ├─ dsp.h
│  │  ├─ features.h
│  │  └─ comms.h
│  ├─ Src/
│  │  ├─ main.c
│  │  ├─ ads131e08.c
│  │  ├─ iepe_frontend.c
│  │  ├─ bridge_frontend.c
│  │  ├─ dsp.c
│  │  ├─ features.c
│  │  ├─ comms.c
│  │  └─ fatfs_logger.c
│  └─ Makefile (optional)
├─ tools/
│  ├─ python/
│  │  ├─ feature_extractor.py
│  │  └─ train_model.py
└─ docs/
   └─ schematics.pdf (hand-drawn / KiCad exported)
```

### 4.1 ads131e08.c (driver) — key points

- Configure SPI to read frames of length `FRAME_BYTES`.
- Use DRDY EXTI to trigger 1 frame read (CS low, SPI DMA rx start, wait for complete callback, CS high).
- Parse 24-bit signed samples into int32_t / float, apply sign extension.

**Important snippet (parsing 24-bit)**

```c
static inline int32_t parse24(const uint8_t *b) {
  int32_t v = (b[0] << 16) | (b[1] << 8) | b[2];
  if (v & 0x800000) v |= 0xFF000000; // sign extend
  return v;
}
```

### 4.2 DMA handling pattern

- Use two buffers: `uint8_t spi_buf[2][FRAME_BYTES];`
- Configure SPI RX DMA circular or use half-transfer/full-transfer interrupts.

- On DMA half-complete, push pointer to DSP queue; on full-complete push other pointer.

### 4.3 DSP task & CMSIS-DSP (dsp.c)

- Use `arm_rfft_fast_f32` for real FFT.
- Convert int32_t raw values to float: `float g = ((float)raw) * scale;` where `scale = Vref / (2^23-1) / gain`.
- Hann window multiply, perform RFFT, magnitude, band energies.

Key function: compute RMS, kurtosis

```
float compute_rms(const float *x, uint32_t N) {
  float sum = 0; for (uint32_t i=0;i<N;i++) sum += x[i]*x[i];
  return sqrtf(sum / (float)N);
}

float compute_kurtosis(const float *x, uint32_t N) {
  float mean = arm_mean_f32(x, N);
  float m2=0,m4=0;
  for (uint32_t i=0;i<N;i++) { float d = x[i]-mean; m2 += d*d; m4 += d*d*d*d; }
  m2 /= N; m4 /= N;
  if (m2==0) return 0; return m4 / (m2*m2) - 3.0f; // excess kurtosis
}
```

### 4.4 Feature extraction (features.c)

- `feature_vector` struct holds RMS, kurtosis, crest, band_energy array, spectral centroid, dominant_freq.
- Envelope demod: use Hilbert transform approximation or analytic envelope via absolute value of bandpass-filtered signal followed by lowpass.

### 4.5 FreeRTOS tasks example

- `Task_Acquire` (highest priority): Waits on DRDY semaphore; arms DMA, on DMA completion pushes raw buffer to `queue_proc`.
- `Task_Process` (medium): Receives raw buffer, converts to floats, windows, computes FFT & features, pushes feature vector to `queue_features` and writes summaries.
- `Task_Anomaly` (low): Runs ML inference on feature vectors, updates health scores, triggers alarms.

### 4.6 Example `main.c` flow

- Hardware init
- Start FreeRTOS scheduler
- Create queues and tasks
- Enable EXTI for DRDY

---

## 5. Sample CSV file (channel map + config)

Below is a CSV you can save as `device_config_channels.csv` and load on device or use in training pipelines.

```
channel_id,channel_type,sensor,adc_channel,fs,unit,gain,notes
acc_brg1_x,vibration,IIS3DWB,AIN0,8000,mg,1,IEPE, mounted on bearing housing X-axis
acc_brg1_y,vibration,IIS3DWB,AIN1,8000,mg,1,IEPE, mounted on bearing housing Y-axis
strain_hook,bridge,foil_bridge,AIN2,200,strain,500,bridge on hook load cell
temp_brg1,temperature,PT100,AIN3,1,C,1,RTD via MAX31865
current_phaseA,current,shunt+AMC1301,AIN4,5000,A,20,isolated shunt via AMC1301
```

Save as CSV and load via simple parser that maps `adc_channel` -> ADS131E08 channel numbers.

---

## 6. Example code excerpts (full code is in the canvas repo)

- `ads131e08.c` — includes `init()`, `read_frame_dma()`, `parse_frame()`
- `dsp.c` — includes `process_buffer()`, `compute_fft()`, `compute_features()`
- `features.c` — contains `compute_rms`, `compute_kurtosis`, `compute_crest`, `band_energy`
- `main.c` — sets up peripherals and starts RTOS

(Full files are included in this canvas document under `firmware/Src` and `firmware/Inc`.)

---

## 7. BOM (condensed)

- STM32H743ZI dev board (or custom STM32H743 MCU)
- ADS131E08 EVM or custom ADS131E08 footprint + ADS131E08 chip
- LM334 (or equivalent) for IEPE current source
- INA333 (instrumentation amp) for bridge
- REF5025 / ADR4525 for bridge excitation
- OPA376 / OPA2140 for precision op amp stages
- TVS diodes (SMBJ58A), ferrite beads, common-mode choke for sensor inputs
- M12 connectors, industrial microSD, isolated DC/DC converter

---

## 8. Next steps & testing checklist

1. Produce a KiCad schematic and PCB layout using the front-end circuits above (I can draft a KiCad project on request).

2. Order EVMs: ADS131E08 EVM and a couple of IEPE accelerometers (or MEMS accelerometers for initial tests).
3. Flash CubeMX-generated project and test SPI comms with ADS (query device ID, read registers).
4. Test single-channel acquisition and validate ENOB with a signal generator.
5. Record a few minutes of raw vibration from a connected gearbox and run the Python `feature_extractor.py` to validate feature pipeline.

---

*Prepared for Pragvansh Dharap — Crane PM project — version 1.1*

If you want, I can now: - export the CSV as a downloadable file and place it into the repo, - generate a KiCad schematic & PCB files for the analog front-end, - generate the full CubeIDE project files compressed as a zip for download.

Tell me which of those you want next and I'll produce it immediately.